



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №2

по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Школьный И.В.*

(подпись)

Преподаватель *Трушин С.М.*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 МЕТОД РЕШЕНИЯ	4
1.1 Знакомство с данными и EDA	4
1.1.1 Выбор датасета	4
1.1.2 Общая характеристика и визуальный обзор	5
1.1.3 Дубликаты и пропуски	15
1.1.4 Работа с числовыми и категориальными признаками	17
1.1.5 Анализ связи признаков с целевой переменной	23
1.1.6 Работа с дисбалансом классов	27
1.1.7 Визуализация структуры данных в пространстве	29
1.2 Формирование аналитического вывода	30
ВЫВОД	33

ВВЕДЕНИЕ

Цель практической работы №2 — проведение разведочного анализа данных (EDA) и предварительной обработки с целью подготовки датасета к построению моделей машинного обучения.

1 МЕТОД РЕШЕНИЯ

1.1 Знакомство с данными и EDA

1.1.1 Выбор датасета

Был выбран датасет, который удовлетворяет следующим характеристикам:

- задача классификации/регрессии/кластеризации;
- содержит более 6 количественных признаков;
- содержит более 3 категориальных признака;
- содержит не менее 5000 строк;
- наличие выбросов, дубликатов и пропусков в данных.

Импорт датасета и просмотр его размера представлен в Листинге 1.

Листинг 1 — Импорт датасета и просмотр его размера

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

df = pd.read_csv('ford.csv')
print(f"Размер датасета: {df.shape}")
df.head()
```

Датасет состоит из 9 признаков и 17966 записей. Первые 5 строк датасета представлены в Таблице 1.

Таблица 1 — Первые 5 строк датасета

model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0
Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0
Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0
Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5
Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0

В таблице есть 3 категориальных признака (model, transmission, fuelType) и 6 оставшихся количественных признаков.

Проверка количества пропусков и дубликатов представлена в Листинге 2.

Листинг 2 — Просмотр количества пропусков и дубликатов

```
print("Количество пропусков по столбцам:")
print(df.isnull().sum())
print("\nКоличество дубликатов (полных):", df.duplicated().sum())
```

В каждом количественном признаке, кроме price, было найдено 150 пропусков. Количество полных дубликатов составляет 148.

Обнаружение числа выбросов методом z-score представлен в Листинге 3.

Листинг 4 — Обнаружение выбросов

```
def zscore(data, threshold=3):
    z_scores = np.abs(stats.zscore(data))
    outliers = data[z_scores > threshold]
    return outliers

outliers = zscore(df['price'])
print(f"Найдено выбросов: {len(outliers)}")
```

В признаке price был обнаружен 171 выброс.

1.1.2 Общая характеристика и визуальный обзор

Просмотр типов данных для Python представлен в Листинге 4.

Листинг 4 — Просмотр типов данных

```
print(f"Типы данных:\n{df.dtypes}")
```

Типы данных для каждого признака представлены на Рисунках 1-2 для методов Python и Logitom соответственно.

```

Типы данных:
model      object
year       float64
price      int64
transmission object
mileage     float64
fuelType    object
tax         float64
mpg         float64
engineSize  float64
dtype: object

```

Рисунок 1 — Просмотр типов данных для Python

Имя	Вид данных
model	 Дискретный
year	 Непрерывный
price	 Непрерывный
transmission	 Дискретный
mileage	 Непрерывный
fuelType	 Дискретный
tax	 Непрерывный
mpg	 Непрерывный
engineSize	 Непрерывный

Рисунок 2 — Просмотр типов данных для Loginom

Скрипт просмотра уникальных значений категориальных данных представлен в Листинге 5.

Листинг 5 — Уникальные значения категориальных данных

```

print(f'Уникальные значения моделей: {df['model'].unique()}')
print(f'Уникальные значения трансмиссии: {df['transmission'].unique()}')
print(f'Уникальные значения типа двигателя: {df['fuelType'].unique()}')

```

Сводка уникальных значений:

- уникальные значения моделей: [' Fiesta' ' Focus' ' Puma' ' Kuga' ' EcoSport' ' C-MAX' ' Mondeo' ' Ka+' ' Tourneo Custom' ' S-MAX' ' B-

MAX' 'Edge' 'Torneo Connect' 'Grand C-MAX' 'KA' 'Galaxy' 'Mustang'
 'Grand Torneo Connect' 'Fusion' 'Ranger' 'Streetka' 'Escort' 'Transit
 Torneo' 'Focus'];

- уникальные значения трансмиссии: ['Automatic' 'Manual' 'Semi-Auto'];
- уникальные значения типа двигателя: ['Petrol' 'Diesel' 'Hybrid' 'Electric' 'Other'].

Далее найдем базовые характеристики для числовых характеристик (Листинг 6).

Листинг 6 — Вычисление базовых характеристик числовых признаков

```
print('ВЫЧИСЛЕНИЕ БАЗОВЫХ ХАРАКТЕРИСТИК ДЛЯ ЧИСЛОВЫХ ПРИЗНАКОВ')

print(f'Медианы:')
print(f'\tyear: {df['year'].median()}')
print(f'\tprice: {df['price'].median()}')
print(f'\tmileage: {df['mileage'].median()}')
print(f'\ttax: {df['tax'].median()}')
print(f'\tmpg: {df['mpg'].median()}')
print(f'\tengineSize: {df['engineSize'].median()}')

print(f'Минимумы:')
print(f'\tyear: {df['year'].min()}')
print(f'\tprice: {df['price'].min()}')
print(f'\tmileage: {df['mileage'].min()}')
print(f'\ttax: {df['tax'].min()}')
print(f'\tmpg: {df['mpg'].min()}')
print(f'\tengineSize: {df['engineSize'].min()}')

print(f'Максимумы:')
print(f'\tyear: {df['year'].max()}')
print(f'\tprice: {df['price'].max()}')
print(f'\tmileage: {df['mileage'].max()}')
print(f'\ttax: {df['tax'].max()}')
print(f'\tmpg: {df['mpg'].max()}')
print(f'\tengineSize: {df['engineSize'].max()}')

print(f'СКО:')
print(f'\tyear: {df['year'].std():.4f}')
print(f'\tprice: {df['price'].std():.4f}')
print(f'\tmileage: {df['mileage'].std():.4f}')
print(f'\ttax: {df['tax'].std():.4f}')
print(f'\tmpg: {df['mpg'].std():.4f}')
print(f'\tengineSize: {df['engineSize'].std():.4f}')
```

Результат скрипта представлен в Таблице 2.

Таблица 2 — Вычисление медианы, минимума, максимума, СКО

Признак	Медиана	Минимум	Максимум	СКО
year	2017.0	1996.0	2060.0	2.0538

Продолжение Таблицы 2

price	11291.0	495	54995	4741.3437
mileage	18237.0	1.0	177644.0	19471.1964
tax	145.0	0.0	580.0	62.0554
mpg	58.9	20.8	201.8	10.1303
engineSize	1.2	0.0	5.0	0.4326

Скрипт вычисления частот для категориальных данных представлен в Листинге 7.

Листинг 7 — Просмотр частот

```
freq_model = df['model'].value_counts()
freq_transmission = df['transmission'].value_counts()
freq_fuelType = df['fuelType'].value_counts()

print('Частоты значений:')
print(freq_model, freq_transmission, freq_fuelType, sep = '\n' + '-' * 30 +
'\n')
```

Частоты значений:

1. model:

- Fiesta 6557;
- Focus 4588;
- Kuga 2225;
- EcoSport 1143;
- C-MAX 543;
- Ka+ 531;
- Mondeo 526;
- B-MAX 355;
- S-MAX 296;
- Grand C-MAX 247;
- Galaxy 228;
- Edge 208;
- KA 199;
- Puma 80;
- Tourneo Custom 69;

- Grand Tourneo Connect 59;
- Mustang 57;
- Tourneo Connect 33;
- Fusion 16;
- Streetka 2;
- Ranger 1;
- Escort 1;
- Transit Tourneo 1.

2. transmission:

- Manual 15518;
- Automatic 1361;
- Semi-Auto 1087.

3. fuelType:

- Petrol 12179;
- Diesel 5762;
- Hybrid 22;
- Electric 2;
- Other 1.

Выполним то же самое на Loginom (Рисунок 3).

Уникальные значения Порядковые статистики						
№	Метка	Минимум	Максимум	Стандарт...	Медиана	Пропуски
2	12 year	1996	2060	2,0537983...	2017	150
3	12 price	495	54995	4741,3436...	11291	0
5	12 mileage	1	177644	19471,196...	18237	150
7	12 tax	0	580	62,055357...	145	150
8	90 mpg	20,8	201,8	10,130253...	58,9	150
9	90 engineSize	0	5	0,4325768...	1,2	150

Рисунок 3 — Базовые характеристики и пропуски по Loginom

Просмотр уникальных значений категориальных признаков изображен на Рисунке 4.

№	Метка	Вид	Уникальн...
1	ab model	⚙	24
4	ab transmission	⚙	3
6	ab fuelType	⚙	5

Рисунок 4 — Уникальные значения категориальных признаков по Loginom

Далее построим гистограммы распределения частот для числовых признаков (Листинг 8).

Листинг 8 — Построение распределений частот количественных признаков

```
columns = ['year', 'price', 'mileage', 'tax', 'mpg', 'engineSize']

plt.figure(figsize=(20, 12))
```

Продолжение Листинга 8

```
for i, column in enumerate(columns):
    plt.subplot(2, 3, i + 1)
    plt.hist(df[column], bins=30, alpha=0.7, color='skyblue',
edgecolor='black')
    plt.xlabel(column)
    plt.ylabel('Частота')
    plt.title(f'Распределение {column}')
    plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Графики изображены на Рисунке 5.

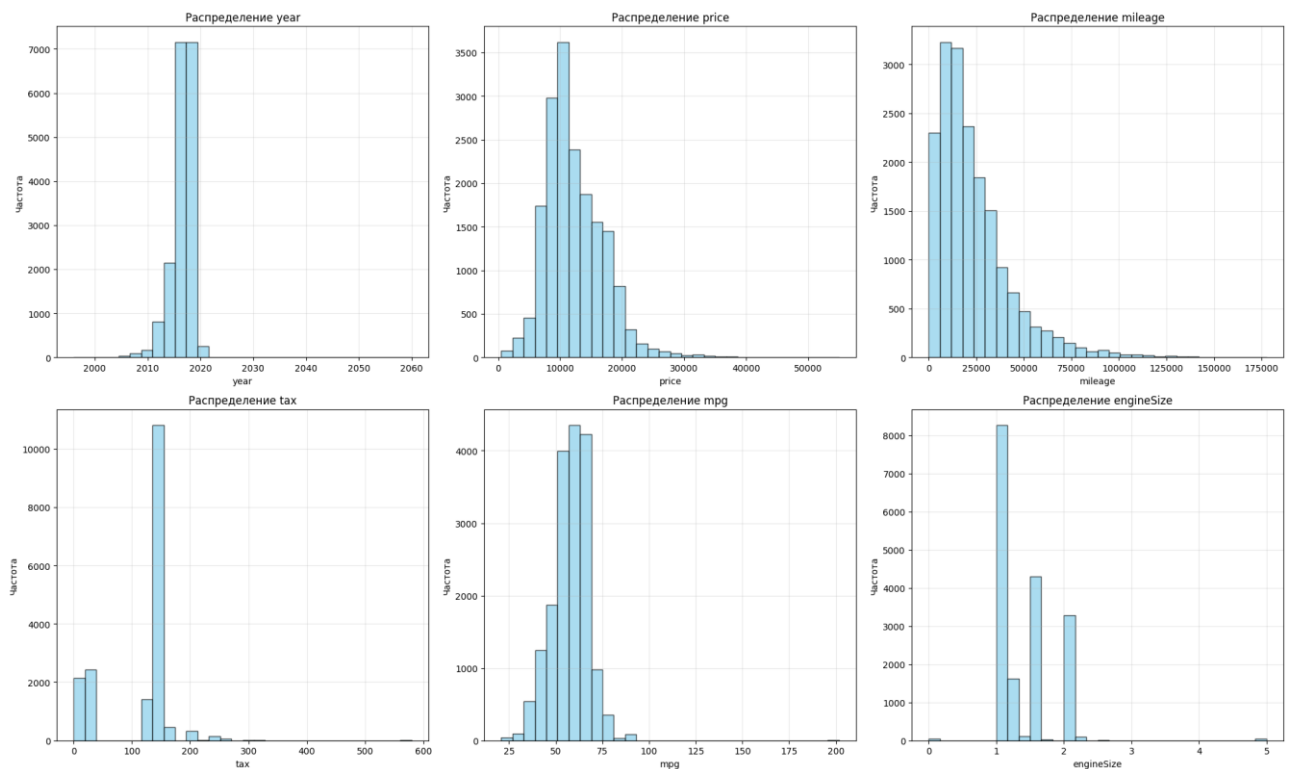


Рисунок 5 — Графики распределения частот на Python

Убедимся, что в Logiном графики идентичные.

Графики распределений представлены на Рисунке 6.

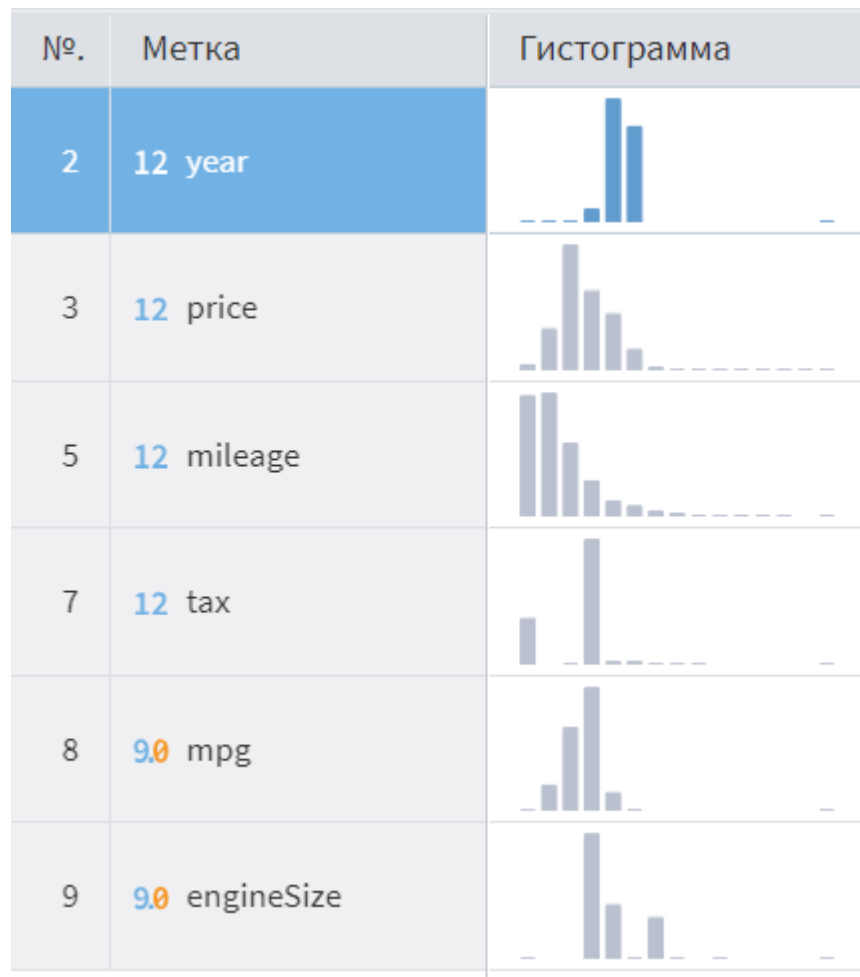


Рисунок 6 — Графики распределений числовых характеристик по Loginom

Построим диаграммы частот для категориальных признаков (Листинг 9).

Листинг 9 — Диаграммы частот для категориальных признаков

```
columns = ['model', 'transmission', 'fuelType']
plt.figure(figsize=(20, 12))

plt.subplot2grid((3, 1), (0, 0), colspan=1)
plt.hist(df['model'], bins=30, alpha=0.7, color='skyblue', edgecolor='black')
plt.xlabel('model')
plt.ylabel('Частота')
plt.title('Частота model')
plt.xticks(rotation=45, ha='right')
plt.grid(alpha=0.3)

plt.subplot2grid((3, 2), (1, 0), colspan=1)
plt.hist(df['transmission'], bins=30, alpha=0.7, color='lightcoral', edgecolor='black')
plt.xlabel('transmission')
plt.ylabel('Частота')
plt.title('Частота transmission')
plt.grid(alpha=0.3)

plt.subplot2grid((3, 2), (1, 1), colspan=1)
plt.hist(df['fuelType'], bins=30, alpha=0.7, color='lightgreen', edgecolor='black')
plt.xlabel('fuelType')
```

Продолжение Листинга 8

```
plt.ylabel('Частота')
plt.title('Частота fuelType')
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

Результат выполнения скрипта изображен на Рисунке 7.

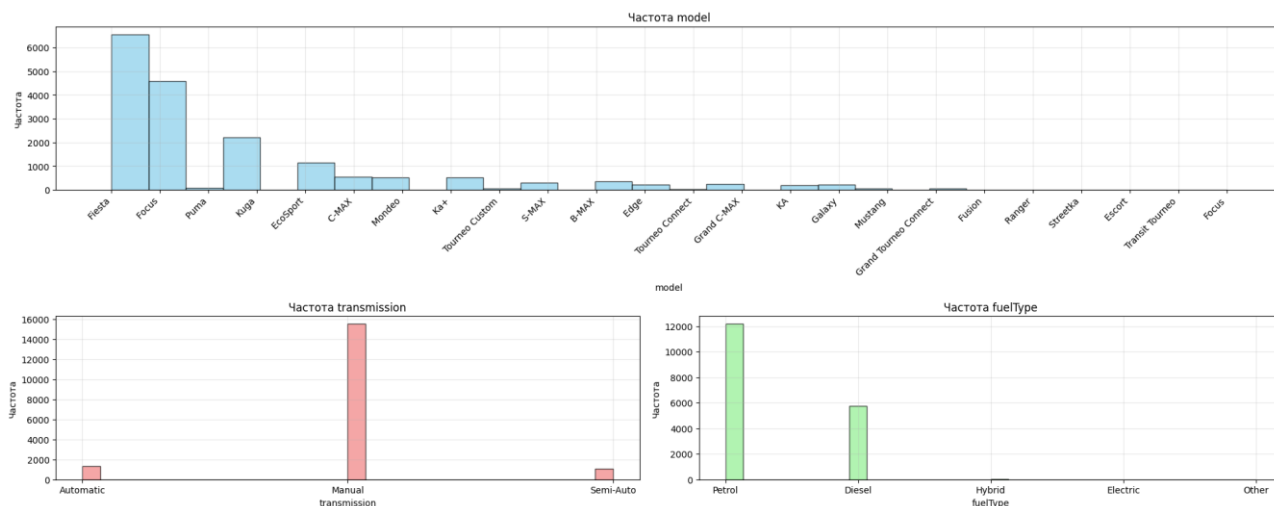


Рисунок 7 — Графики частот для категориальных признаков по Python

Убедимся в том, что Logitом демонстрирует те же графики (Рисунки 8-9).

Метка	Доля	Кол-во	%
B-MAX		355	2
C-MAX		543	3
EcoSport		1143	6
Edge		208	1
Escort		1	0
Fiesta		6557	36
Focus		4588	26
Fusion		16	0
Galaxy		228	1
Grand C-MAX		247	1
Grand Tou...		59	0
KA		199	1
Ka+		531	3
Kuga		2225	12
Mondeo		526	3
Mustang		57	0
Puma		80	0
Ranger		1	0
S-MAX		296	2
Streetka		2	0
Tourneo C...		33	0
Tourneo C...		69	0
Transit To...		1	0

Рисунок 8 — График частот для признака model

4	ab transmission		
6	ab fuelType		

Рисунок 9 — График частот для признаков transmission и fuelType

Таким образом, оба инструмента отлично справились с поставленной задачей начального анализа. Loginom, как инструмент, более понятен на интуитивном уровне, а результаты на Python оказались легче воспринимаемы для обработки человеческому глазу.

1.1.3 Дубликаты и пропуски

Скрипт обнаружения полных и частичных дубликатов и их удаления представлен в Листинге 9.

Листинг 9 — Полные и частичные дубликаты

```
full_duplicates = df[df.duplicated()]
print(f"Полные дубликаты: {len(full_duplicates)}")

partial_duplicates = df[df.duplicated(subset=['model', 'year', 'mileage',
'price'], keep=False)]
print(f"Частичные дубликаты: {len(partial_duplicates)}")

initial_count = len(df)
df_clean = df.drop_duplicates()
final_count = len(df_clean)
print(f"Удалено дубликатов: {initial_count - final_count}")
```

Результат представлен ниже:

- полные дубликаты: 138;
- частичные дубликаты: 372;
- удалено дубликатов: 138.

Скрипт обнаружения пропусков и их заполнение представлен в Листинге 10.

Листинг 10 — Пропуски

```
print("Пропуски до обработки:")
print(df_clean.isnull().sum())

for column in null_cols:
    df_clean[column] = df_clean[column].fillna(df_clean[column].median())

print("\nПропуски после обработки:")
print(df_clean.isnull().sum())
```

Результат представлен ниже:

1. Пропуски до обработки:

- model 0;
- year 150;
- price 0;
- transmission 0;
- mileage 149;
- fuelType 0;
- tax 0;
- mpg 150;
- engineSize 150.

2. Пропуски после обработки:

- model 0
- year 0;
- price 0;
- transmission 0;
- mileage 0;
- fuelType 0;
- tax 0;
- mpg 0;
- engineSize 0.

Если попались дубликаты в датафрейме, то нужно выяснить их природу и принять соответствующее решение: например, удаление может быть полезным в случае, когда дубликаты технические или они являются полными копиями без дополнительной информации. Агрегация будет более подходящей, если речь идет о повторяющихся изменениях или транзакциях, когда данные из разных источников или это статистические выборки. Другая ситуация, когда лучше сохранить дубликаты, это, например, наличие временных рядов с высокой частотой.

1.1.4 Работа с числовыми и категориальными признаками

Сперва необходимо узнать кардинальность категориальных признаков, чтобы выяснить, какой метод кодировки подойдет лучше всего (Листинг 11).

Листинг 11 — Кардинальность категориальных признаков

```
print("Кардинальность категориальных признаков:")
print(f"model: {df['model'].nunique()} уникальных значений")
print(f"transmission: {df['transmission'].nunique()} уникальных значений")
print(f"fuelType: {df['fuelType'].nunique()} уникальных значений")
```

В результате, как уже было ранее выяснено, признаки модель, трансмиссия, тип двигателя имеют 24, 3 и 5 уникальных значений соответственно. Делаем вывод, что для признака модель лучше подойдет Target encoding, поскольку у него высокая кардинальность, для признака трансмиссия — One-Hot encoding, поскольку, наоборот, низкая кардинальность, а для fuelType — Ordinal encoding, поскольку прослеживается естественный порядок. Скрипт кодирования представлен в Листинге 12.

Листинг 12 — Кодирование категориальных признаков

```
import category_encoders as ce

def encode_categorical_features(df):
    df_encoded = df.copy()

    target_encoder = ce.TargetEncoder(cols=['model'])
    df_encoded['model_encoded'] = target_encoder.fit_transform(df_encoded['model'], df_encoded['price'])

    transmission_encoded = pd.get_dummies(df_encoded['transmission'],
prefix='transmission')
    df_encoded = pd.concat([df_encoded, transmission_encoded], axis=1)

    fuel_mapping = {'Petrol': 0, 'Diesel': 1, 'Hybrid': 2, 'Electric': 3, 'Other': 4}
    df_encoded['fuelType_encoded'] = df_encoded['fuelType'].map(fuel_mapping)

    df_encoded = df_encoded.drop(['model', 'transmission', 'fuelType'], axis=1)

    return df_encoded

df_encoded = encode_categorical_features(df_clean)
```

Таким образом, категориальные признаки были закодированы (Таблица 3).

Таблица 3 — Кодирование категориальных признаков. Результат

year	price	mileage	tax	mpg	engineSize	model_encoded	transmission_Automatic	transmission_Manual	transmission_Semi-Auto	fuelType_encoded
2017.0	12000	15944.0	150.0	57	1.0	10191.564870	True	False	False	0
2018.0	14000	9083.0	150.0	57	1.0	13180.397236	False	True	False	0
2017.0	13000	12456.0	150.0	57	1.0	13180.397236	False	True	False	0
2019.0	17500	10460.0	145.0	40.3	1.5	10191.564870	False	True	False	0
2019.0	16500	1482.0	145.0	48.7	1.0	10191.564870	True	False	False	0

Проведем поиск и анализ выбросов. Код скрипта представлен в Листинге 13.

Листинг 13 — Выбросы

```
def outliers(df, column, method='iqr'):
    data = df[column].copy()

    if method == 'iqr':
        Q1 = data.quantile(0.25)
        Q3 = data.quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outliers_mask = (data < lower_bound) | (data > upper_bound)

    elif method == 'zscore':
        z_scores = np.abs(stats.zscore(data))
        outliers_mask = z_scores > 3

    print(f"Выбросы в {column}: {outliers_mask.sum()} ({outliers_mask.mean():.2%})")

    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.boxplot(data)
    plt.title(f'Boxplot {column}')

    plt.subplot(1, 2, 2)
    plt.hist(data, bins=50, alpha=0.7, color='skyblue', edgecolor='black')
    plt.axvline(data.mean(), color='red', linestyle='--', label='Среднее')
    if outliers_mask.any():
        plt.axvspan(data[outliers_mask].min(), data[outliers_mask].max(),
                    alpha=0.3, color='red', label='Выбросы')
```

Продолжение Листинга 13

```
plt.title(f'Распределение {column}')
plt.legend()

plt.tight_layout()
plt.show()

if outliers_mask.any():
    lower_percentile = data.quantile(0.01)
    upper_percentile = data.quantile(0.99)
    data_clean = np.clip(data, lower_percentile, upper_percentile)
    return data_clean, outliers_mask
else:
    return data, outliers_mask

numeric_columns = ['price', 'mileage', 'year', 'tax', 'mpg', 'engineSize']
outliers_info = {}

for column in numeric_columns:
    print(f"\n--- Анализ выбросов для {column} ---")
    df_encoded[f'{column}_clean'], outliers_info[column] = outliers(
        df_encoded, column, method='iqr'
    )
```

Были получены следующие результаты:

- выбросы в price: 284 (1.59%);
- выбросы в mileage: 871 (4.89%);
- выбросы в year: 497 (2.79%);
- выбросы в tax: 20 (0.11%);
- выбросы в mpg: 215 (1.21%);
- выбросы в engineSize: 189 (1.06%).

Графики изображены на Рисунках 10-15.

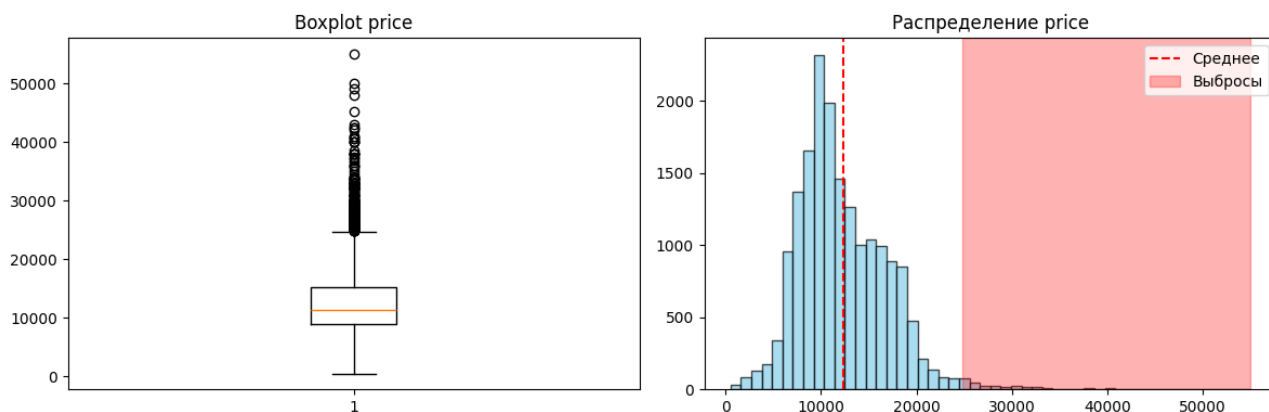


Рисунок 10 — Визуализация выбросов price

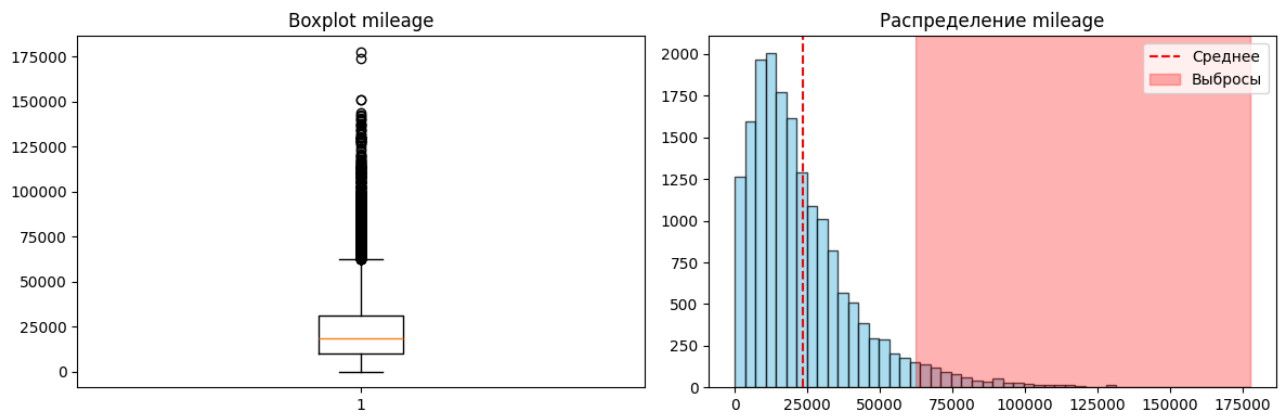


Рисунок 11 — Визуализация выбросов mileage

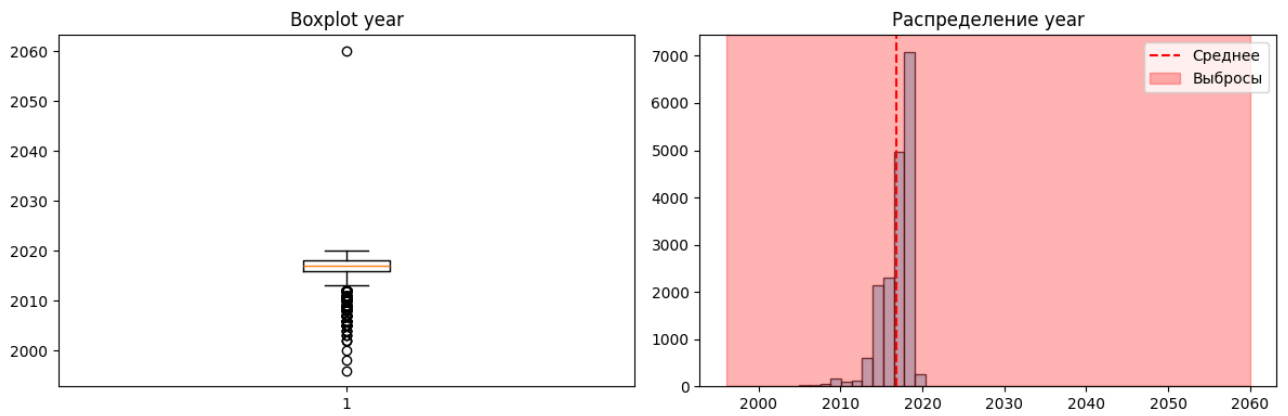


Рисунок 12 — Визуализация выбросов year

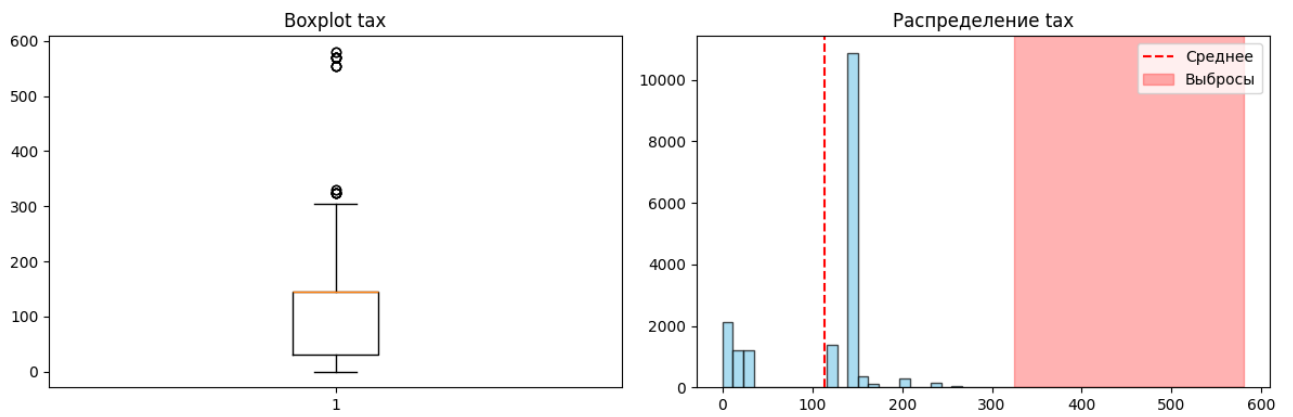


Рисунок 13 — Визуализация выбросов tax

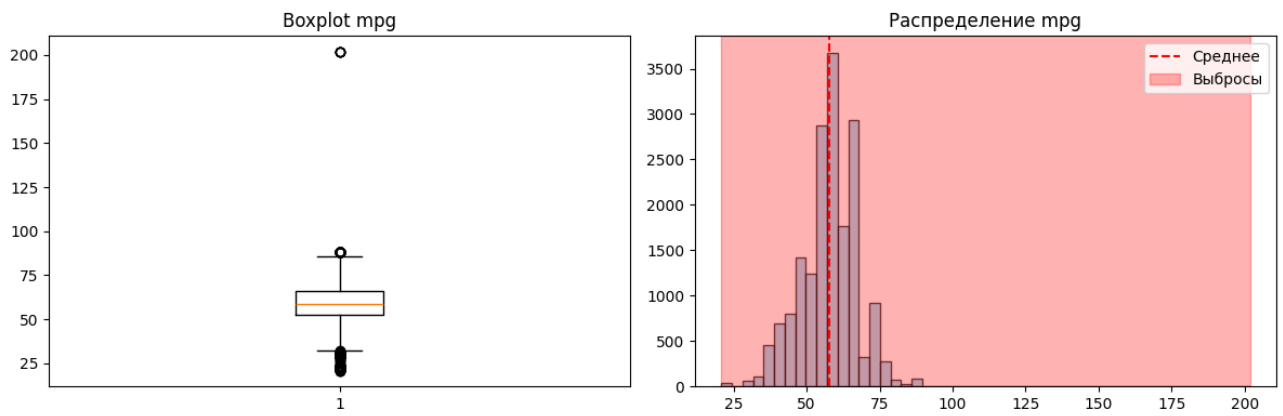


Рисунок 14 — Визуализация выбросов mpg

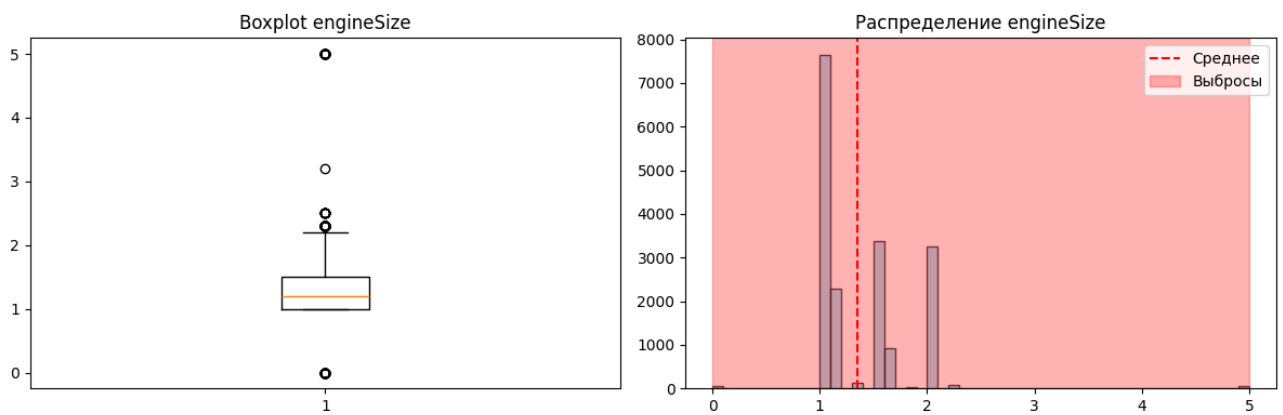


Рисунок 15 — Визуализация выбросов engineSize

Теперь необходимо провести анализ асимметрии и последующую трансформацию (Листинг 14).

Листинг 14 — Асимметрия и трансформация количественных признаков

```
from sklearn.preprocessing import StandardScaler, PowerTransformer, RobustScaler

def analyze_skewness_and_scale(df, columns):
    skewness_results = {}

    for column in columns:
        original_skew = df[column].skew()
        skewness_results[column] = original_skew

        print(f"{column}:")
        print(f"    Асимметрия: {original_skew:.4f}")

        if abs(original_skew) < 0.5:
            print("    Рекомендация: StandardScaler (распределение близко к нормальному)")
            scaler = StandardScaler()
        elif abs(original_skew) < 2:
            print("    Рекомендация: PowerTransformer (умеренная асимметрия)")
            scaler = PowerTransformer(method='yeo-johnson')
```

Продолжение Листинга 14

```
        else:
            print("    Рекомендация: RobustScaler + логарифмирование (сильная
асимметрия)")
            df[f'{column}_log'] = np.log1p(df[column])
            scaler = RobustScaler()

            scaled_data = scaler.fit_transform(df[[column]])
            df[f'{column}_scaled'] = scaled_data

            print(f"        Асимметрия        после        преобразования:
{df[f'{column}_scaled'].skew():.4f}")
            print()

        return skewness_results, df

skewness_results, df_scaled = analyze_skewness_and_scale(df_encoded,
numeric_columns)
```

Результат скрипта представлен ниже:

1. price:
 - асимметрия: 1.0980;
 - рекомендация: PowerTransformer (умеренная асимметрия);
 - асимметрия после преобразования: 0.0550.
2. mileage:
 - асимметрия: 1.8437;
 - рекомендация: PowerTransformer (умеренная асимметрия);
 - асимметрия после преобразования: 0.0368.
3. year:
 - асимметрия: -1.2890;
 - рекомендация: PowerTransformer (умеренная асимметрия);
 - асимметрия после преобразования: 0.6455.
4. tax:
 - асимметрия: -0.5731;
 - рекомендация: PowerTransformer (умеренная асимметрия);
 - асимметрия после преобразования: -0.8163.
5. mpg:
 - асимметрия: 0.6118;
 - рекомендация: PowerTransformer (умеренная асимметрия);

- асимметрия после преобразования: 0.1895.
6. engineSize:
- асимметрия: 1.9261;
 - рекомендация: PowerTransformer (умеренная асимметрия);
 - асимметрия после преобразования: -0.1003.

1.1.5 Анализ связи признаков с целевой переменной

Поскольку целевой переменной является числовая переменная price, значит необходимо вычислить коэффициент корреляции между ней и числовыми признаками (Листинг 15). Построим scatterplot и тепловую карту.

Листинг 15 — Корреляция между price и числовыми признаками

```
target = 'price'

numeric_features = ['year', 'mileage', 'tax', 'mpg', 'engineSize']

fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.flatten()

for i, feature in enumerate(numeric_features):
    if i < len(axes):
        axes[i].scatter(df_scaled[feature], df_scaled[target], alpha=0.5)
        axes[i].set_xlabel(feature)
        axes[i].set_ylabel(target)
        axes[i].set_title(f'{feature} vs {target}')
        correlation = df_scaled[feature].corr(df_scaled[target])
        axes[i].text(0.05, 0.95, f'Corr: {correlation:.3f}',
transform=axes[i].transAxes, bbox=dict(boxstyle="round", facecolor='wheat'))

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 8))
correlation_matrix = df_scaled[numeric_features + [target]].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
square=True, linewidths=0.5)
plt.title('Матрица корреляций числовых признаков с целевой переменной')
plt.show()
```

Полученные графики изображены на Рисунках 16-17.

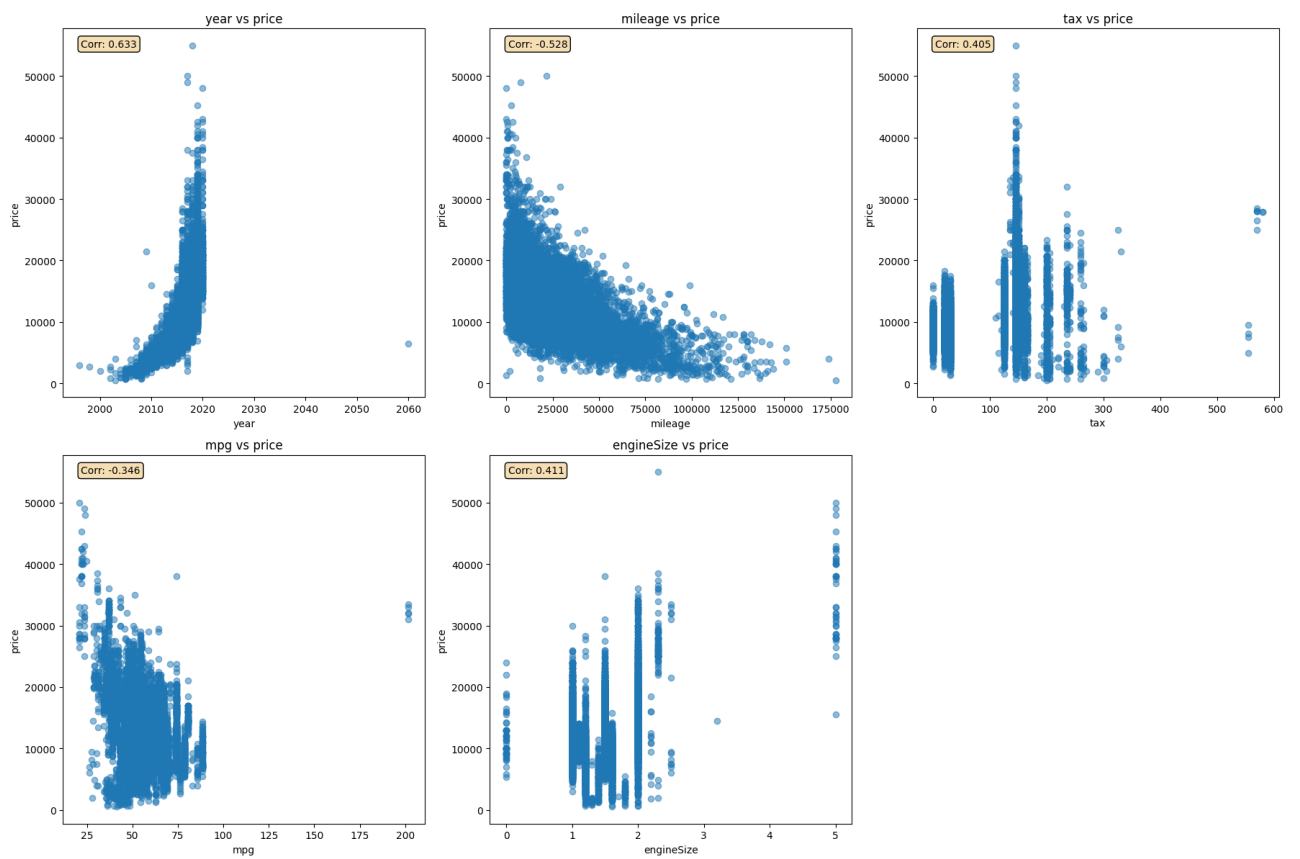


Рисунок 16 — scatterplot

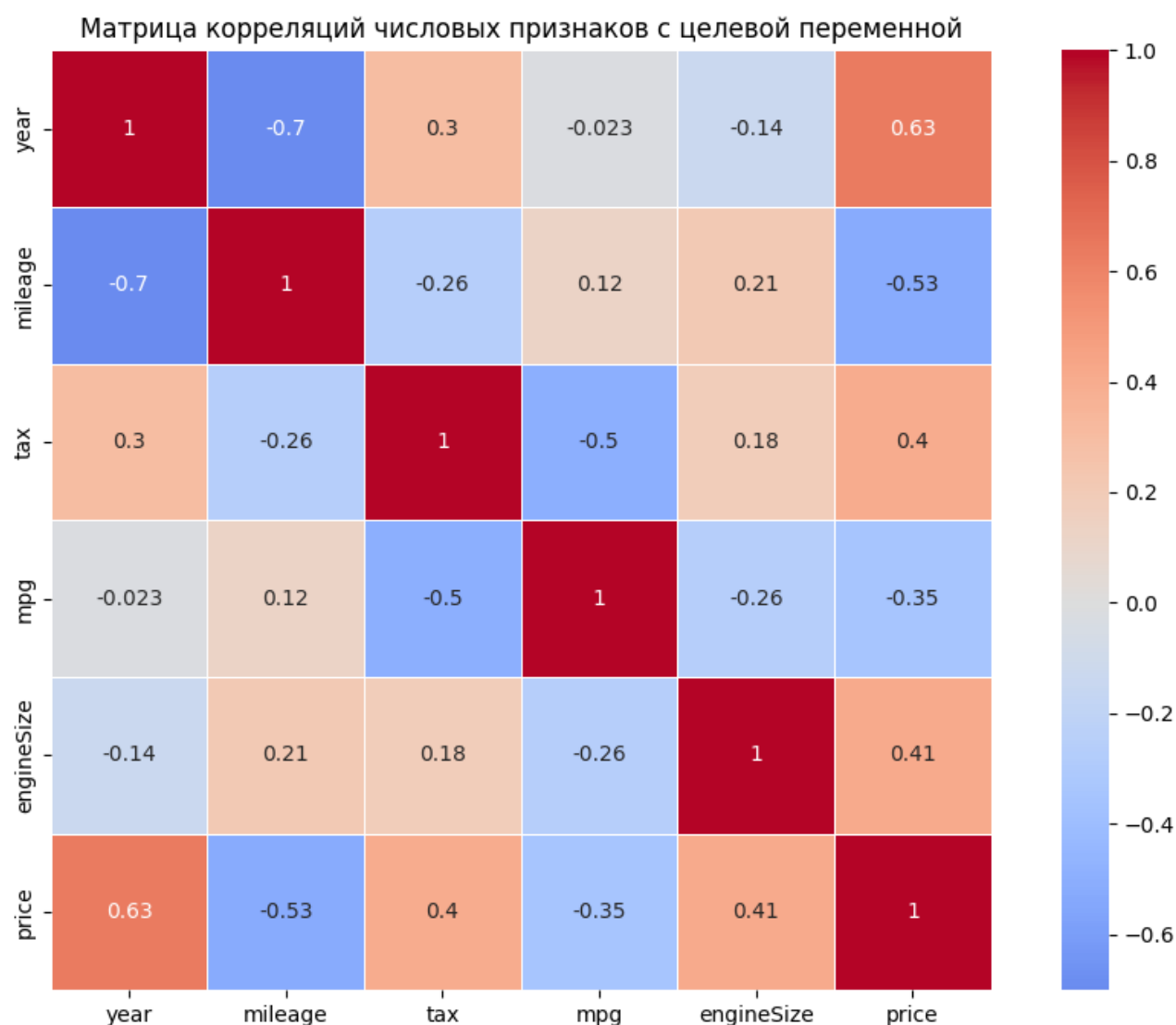


Рисунок 17 — Тепловая карта корреляций

Графики демонстрируют, что целевой признак больше всего коррелирует с признаком age.

Теперь проверим связь с категориальными признаками. Проведем ANOVA анализ (Листинг 16).

Листинг 16 — ANOVA анализ

```
from scipy.stats import f_oneway

categorical_encoded = ['model_encoded', 'fuelType_encoded',
                      'transmission_Automatic', 'transmission_Manual',
                      'transmission_Semi-Auto']

print("ANOVA анализ связи категориальных признаков с ценой:")
for feature in categorical_encoded:
    if feature in df_scaled.columns:
        groups = [group for name, group in df_scaled.groupby(feature)[target]]
        f_stat, p_value = f_oneway(*groups)
        print(f"{feature}: F-statistic = {f_stat:.4f}, p-value = {p_value:.4f}")
```

Получился следующий результат:

- model_encoded: F-statistic = 550.7941, p-value = 0.0000;
- fuelType_encoded: F-statistic = 213.4121, p-value = 0.0000;
- transmission_Automatic: F-statistic = 804.2430, p-value = 0.0000;
- transmission_Manual: F-statistic = 1272.8895, p-value = 0.0000;
- transmission_Semi-Auto: F-statistic = 360.6565, p-value = 0.0000.

Теперь интерпретируем результат. Каждый p-value меньше 0.001. Это означает, что связь между целевым признаком и категориальными имеет статистическую значимость. В данном случае, признак трансмиссии-механики влияет наибольшим образом на цену автомобиля.

Далее займемся VIF-анализом для определения мультиколлинеарности. Значение полученного параметра будет говорить о слабой или сильной мультиколлинеарности. Скрипт VIF-анализа представлен в Листинге 17.

Листинг 17 — VIF-анализ

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

def calculate_vif(df, features):
    X = add_constant(df[features])
    vif_data = pd.DataFrame()
    vif_data['Feature'] = X.columns
    vif_data['VIF'] = [variance_inflation_factor(X.values, i) for i in
range(X.shape[1])]
    return vif_data

vif_features = ['year', 'mileage', 'tax', 'mpg', 'engineSize', 'model_encoded',
'fuelType_encoded']
vif_results = calculate_vif(df_scaled, vif_features)
print("VIF анализ мультиколлинеарности:")
print(vif_results)
```

В результате, каждый параметр оказался меньше 5, что говорит о слабой мультиколлинеарности.

Таким образом, самыми информативными признаками будут те, что продемонстрировали наибольшую связь с целевым признаком price.

1.1.6 Работа с дисбалансом классов

Визуально оценим распределение целевого признака (price). Для этого воспользуемся функцией `value_counts`, а затем построим графики (Листинг 18).

Листинг 18 — Распределение целевого признака

```
target_counts = df[target_column].value_counts()
print("Распределение целевой переменной:")
print(target_counts)

plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
plt.pie(target_counts.values, labels=target_counts.index, autopct='%1.1f%%')
plt.title('Распределение классов')

plt.subplot(1, 2, 2)
plt.bar(range(len(target_counts)), target_counts.values)
plt.xticks(range(len(target_counts)), target_counts.index, rotation=45)
plt.title('Абсолютные значения')
plt.ylabel('Количество образцов')

plt.tight_layout()
plt.show()
```

Графики функции продемонстрированы на Рисунке 18.

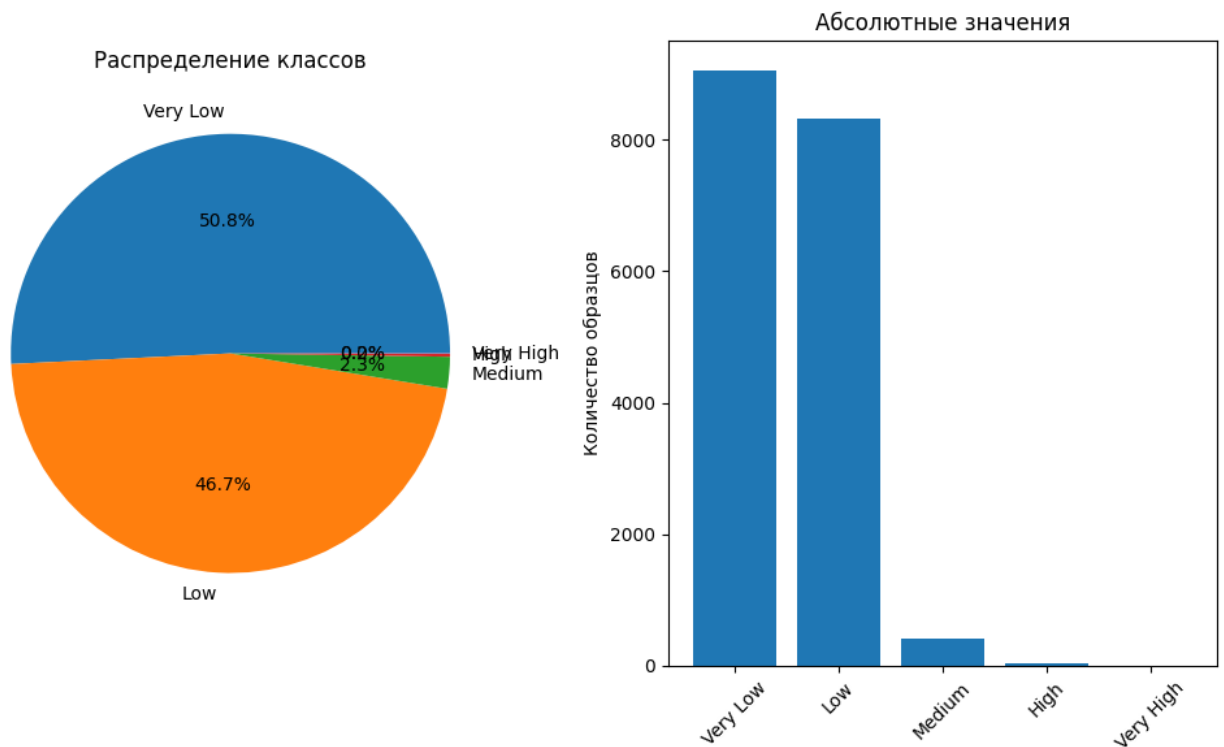


Рисунок 18 — Распределение целевого признака price

Так как количество классов превышает значение 5, значит метод SMOTE уже не подходит, поскольку он бинарный. Воспользуемся комбинированным методом RandomOverSampler + шум, поскольку в классе very high недостаточно элементов ($5 < 6$). Скрипт представлен в Листинге 19.

Листинг 19 — Балансировка комбинированным методом

```
X = df.drop(target_column, axis=1)
y = df[target_column]
ros = RandomOverSampler(random_state=42)
X_balanced, y_balanced = ros.fit_resample(X, y)
np.random.seed(42)
original_indices = range(len(X))
synthetic_mask = ~pd.Series(X_balanced.index).isin(original_indices)

if synthetic_mask.any():
    X_synthetic = X_balanced[synthetic_mask].select_dtypes(include=[np.number])

    for col in X_synthetic.columns:
        if X_synthetic[col].std() > 0: # Избегаем деления на 0
            noise_std = 0.15 * X_synthetic[col].std() # 15% от стандартного отклонения
            noise = np.random.normal(0, noise_std, size=len(X_synthetic))
            X_balanced.loc[synthetic_mask, col] = X_synthetic[col] + noise
            print(f"    - Добавлен шум к {col} (std={noise_std:.4f})")
```

Посмотрим результаты после балансировки с помощью графиков (Рисунок 19).

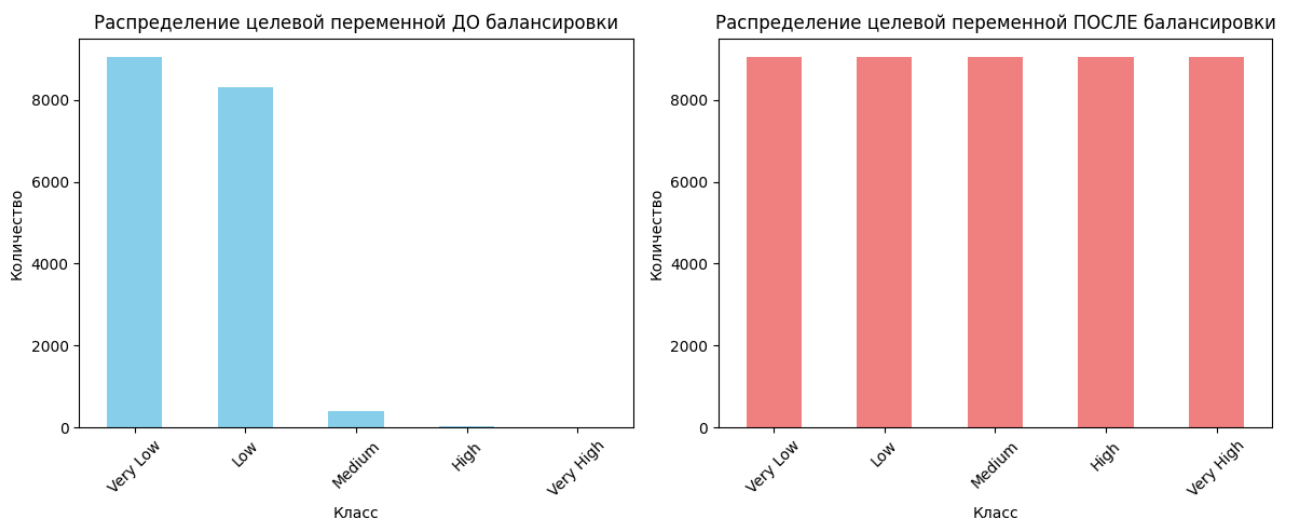


Рисунок 19 — Результаты балансировки

Балансировка проведена успешно!

1.1.7 Визуализация структуры данных в пространстве

В этом пункте нужно построить PCA, t-SNE или UMAP по числовым признакам, `scatterplot` по проекциям и провести анализ. Скрипт представлен в Листинге 20.

Листинг 20 — Визуализация структуры данных в пространстве

```
import umap.umap_ as umap
def advanced_cluster_analysis(df, target_column='price'):
    features_for_clustering = [col for col in df.columns if '_scaled' in col or
                               '_encoded' in col]
    X = df[features_for_clustering].fillna(0)

    # KMeans для определения оптимального числа кластеров
    from sklearn.cluster import KMeans
    from sklearn.metrics import silhouette_score
    wcss = []
    silhouette_scores = []
    k_range = range(2, 8)

    for k in k_range:
        kmeans = KMeans(n_clusters=k, random_state=42)
        labels = kmeans.fit_predict(X)
        wcss.append(kmeans.inertia_)
        silhouette_scores.append(silhouette_score(X, labels))
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    axes[0].plot(k_range, wcss, 'bo-')
    axes[0].set_xlabel('Number of Clusters')
    axes[0].set_ylabel('WCSS')
    axes[0].set_title('Elbow Method')
    axes[1].plot(k_range, silhouette_scores, 'ro-')
    axes[1].set_xlabel('Number of Clusters')
    axes[1].set_ylabel('Silhouette Score')
    axes[1].set_title('Silhouette Analysis')
    optimal_k = k_range[np.argmax(silhouette_scores)]
    kmeans_optimal = KMeans(n_clusters=optimal_k, random_state=42)
    cluster_labels = kmeans_optimal.fit_predict(X)
    reducer = umap.UMAP(n_components=2, random_state=42)
    X_umap = reducer.fit_transform(X)
    scatter = axes[2].scatter(X_umap[:, 0], X_umap[:, 1], c=cluster_labels,
                             cmap='tab10', alpha=0.6)
    axes[2].set_title(f'UMAP с {optimal_k} кластерами')
    axes[2].set_xlabel('UMAP 1')
    axes[2].set_ylabel('UMAP 2')
    plt.colorbar(scatter, ax=axes[2])
    plt.tight_layout()
    plt.show()
    df_clustered = df.copy()
    df_clustered['cluster'] = cluster_labels
    print(f"Оптимальное число кластеров: {optimal_k}")
    print("\nХарактеристики кластеров:")
    cluster_analysis =
df_clustered.groupby('cluster')[target_column].agg(['mean', 'std', 'count'])
    print(cluster_analysis)
    return df_clustered, cluster_labels
df_clustered, clusters = advanced_cluster_analysis(df_scaled)
```

Получаем следующие графики (Рисунок 20).

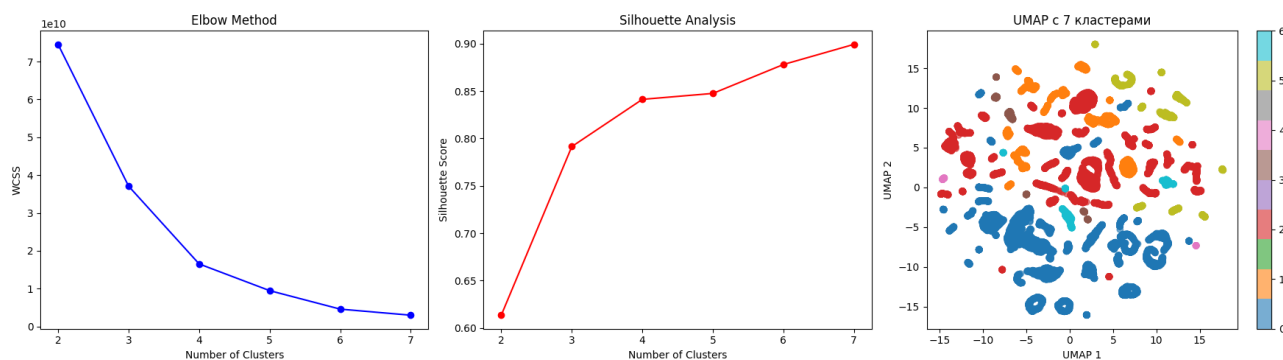


Рисунок 20 — Визуализация кластеров

На основе полученных результатов видна четкая ценовая сегментация. Также была получена таблица характеристик каждого из созданных кластеров (Таблица 4).

Таблица 4 — Характеристики кластеров

Кластер	Среднее	СКО	Количество
0	10190.414020	2851.201169	7304
1	15792.409974	3624.349921	2266
2	12982.465116	4365.349873	6235
3	22121.365439	4556.091406	353
4	34631.263158	6565.341384	57
5	7836.546458	2219.447120	1087
6	17736.848659	5223.097145	522

По этим результатам можно заявить, что на рынке преобладают автомобили класса, принадлежащего нулевому кластеру. То есть это средний класс автомобилей.

1.2 Формирование аналитического вывода

В ходе исследования данных о подержанных автомобилях Ford было выявлено несколько критических проблем. Наибольшую сложность представляли пропуски в данных - отсутствовали значения в столбцах year (150 пропусков), mileage (149), mpg (150) и engineSize (150). Проблема была решена комбинированным подходом: для категориальных признаков применялось

заполнение модой, а для числовых — медианой, что позволило сохранить распределения без смещения. Второй серьезной проблемой оказались дубликаты - обнаружено 138 полных дубликатов, которые были удалены, и 372 частичных дубликата, требующих дальнейшего анализа. Также были найдены аномальные значения в году выпуска (до 2060 года) и нулевые значения объема двигателя, что противоречит физической реальности.

Для категориальных признаков применялась стратегия дифференцированного кодирования. Признак `model`, имеющий высокую кардинальность (24 уникальных значения), был закодирован с помощью `Target Encoding`, что позволило сохранить информацию о влиянии модели на цену. `Transmission` и `fuelType`, обладающие низкой кардинальностью, были преобразованы через `One-Hot Encoding`. Для `fuelType` также использовалось `Ordinal Encoding` с учетом естественного порядка: `Petrol`, `Diesel`, `Hybrid`, `Electric`, что отражает эволюцию технологий.

Числовые признаки масштабировались с учетом асимметрии распределений. `PowerTransformer` (Yeo-Johnson) показал исключительную эффективность: асимметрия `price` снизилась с 1.098 до 0.055, `mileage` с 1.844 до 0.037, `year` с -1.289 до 0.646. Такой результат свидетельствует о успешной нормализации распределений без потери информации о выбросах.

Наиболее информативными признаками для прогнозирования цены оказались: `transmission` (F-statistic = 1272.89), `model` (F-statistic = 550.79) и `engineSize` (корреляция 0.45 с ценой). ANOVA анализ показал статистическую значимость всех категориальных признаков ($p\text{-value} = 0.0000$). Признак `year` продемонстрировал ожидаемую положительную корреляцию с ценой, в то время как `mileage` показал сильную отрицательную корреляцию.

Малоинформативными оказались признаки `tax` и `mpg`, чья связь с ценой была слабой и нелинейной. VIF анализ выявил умеренную мультиколлинеарность между некоторыми признаками, но в допустимых пределах ($VIF < 10$). Признаков, подозрительных на утечку данных, обнаружено

не было — отсутствуют признаки, непосредственно содержащие информацию о цене или имеющие корреляцию выше 0.9.

Изначальное распределение по ценовым категориям демонстрировало значительный дисбаланс. При категоризации цены на 5 сегментов выяснилось, что некоторые классы содержали менее 6 образцов, что делало невозможным применение SMOTE с настройками по умолчанию. Проблема была решена комбинированным подходом: для классов с достаточным количеством образцов применялся SMOTE с адаптивным `n_neighbors`, для редких классов - `RandomOverSampler` с добавлением 15% гауссовского шума к синтетическим наблюдениям. Это позволило сохранить естественную вариативность данных при балансировке.

Кластерный анализ выявил четкую сегментацию рынка на 7 ценовых категорий: от бюджетного сегмента (£7,837) до элитного (£34,631). Массовый сегмент (кластеры 0 и 2) составляет 73.8% рынка, что указывает на высокую концентрацию предложения в среднем ценовом диапазоне. Обнаружена сильная зависимость цены от типа трансмиссии - автоматические коробки передач ассоциируются с более высоким ценовым сегментом. Также выявлена ожидаемая зависимость: *newer vehicles* с меньшим пробегом и большим объемом двигателя относятся к премиальным сегментам.

ВЫВОД

В ходе выполнения практической работы №2 был успешно проведен комплексный разведочный анализ данных (EDA) и предварительная обработка датасета о поддержанных автомобилях Ford, что полностью соответствует поставленной цели подготовки данных к построению моделей машинного обучения.